

Allow pack expansion in lambda init-capture

Contents

- [Motivation](#)
- [Init-capture restriction history](#)
- [Proposal](#)
- [Acknowledgements](#)
- [References](#)

Motivation

With the introduction of generalized lambda capture [1], lambda captures can be nearly arbitrarily complex and solve nearly all problems. However, there is still an awkward hole in the capabilities of lambda capture when it comes to parameter packs: you can only capture packs by copy, by reference, or by... `std::tuple`?

Consider the simple example of trying to wrap a function and its arguments into a callable to be accessed later. If we copy everything, the implementation is both easy to write and read:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    // the capture here can also be just [=]
    return [f, args...]() -> decltype(auto) {
        return std::invoke(f, args...);
    };
}
```

But if we try to be more efficient about the implementation and try to `move` all the arguments into the lambda? It seems like you should be able to use an *init-capture* and write:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    return [f=std::move(f), args=std::move(args)...]() -> decltype(auto) {
        return std::invoke(f, args...);
    };
}
```

But this runs afoul of very explicit wording from [\[expr.prim.lambda.capture\]/17](#), emphasis mine:

*A simple-capture followed by an ellipsis is a pack expansion. An **init-capture** followed by an ellipsis is ill-formed.*

As a result of this restriction, our only option is to put all the `args...` into a `std::tuple`. But once we do that, we don't have access to the arguments as a parameter pack, so we need to pull them back out of the tuple in the body, using something like `std::apply()`:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    return [f=std::move(f), tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) {
        return std::apply(f, tup);
    };
}
```

Which gets even worse if what we wanted to do with that captured parameter pack was invoke a *named* function rather than a captured object. At that point, all semblance of comprehension goes out the window:

By copy	By move
<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [args...]() -> decltype(auto) { return foo(args...); }; }</pre>	<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) { return std::apply([](auto const&... args) -> decltype(auto) { return foo(args...); }, tup); }; }</pre>

We can do better.

Init-capture restriction history

The explicit restriction on pack expansion in *init-capture* was in the initial wording paper for generalized lambda capture [2], due in part to the rules proposed in that paper as to how *init-capture* would work. The original wording read:

For every *init-capture* a non-static data member named by the *identifier* of the *init-capture* is declared in the closure type. This member is not a bit-field and not mutable. The type of that member corresponds to the type of a hypothetical variable declaration of the form "auto *init-capture* ";, except that the variable name (i.e., the *identifier* of the *init-capture*) is replaced by a unique identifier.

Which introduces a problem, as explained by Richard Smith in [3]:

One problem here is that an *init-capture* introduces a **named** member of the closure type. A class member name that names a pack would be a new notion, and would bring with it significant additional complications (such as the inability to determine syntactically whether a construct contains an unexpanded parameter pack).

[...]

Consider this:

```
template <typename T> void call_f(T t) {
    f(t.x...);
}
```

Right now, this is ill-formed (no diagnostic required) because "t.x" does not contain an unexpanded parameter pack. But if we allow class members to be pack expansions, this code could be valid -- we'd lose any syntactic mechanism to determine whether an expression contains an unexpanded pack. This is fatal to at least one implementation strategy for variadic templates. It also admits the possibility of pack expansions occurring outside templates, which current implementations are not well-suited to handle.

Since *init-captures* add named members to the closure type, allowing *init-captures* to be pack expansions risks introducing the same problem if those names are visible in **any** context outside the body of the lambda-expression itself.

However, this problem went away with the adoption of CWG 1760 [4], which changed the wording from *init-capture* introducing a *named* member (of unspecified access) to introducing an *unnamed* member. Once *init-capture* doesn't give us named members, the problem that was pointed out in [3] is no longer a problem. There would be no named pack member to give complications in parsing, so this proposal claims that this restriction is no longer necessary.

Proposal

The proposal is to simply remove the restriction on pack expansions in *init-capture* in [expr.prim.lambda.capture]/17:

A *simple-capture* or *init-capture* followed by an ellipsis is a pack expansion. ~~An *init-capture* followed by an ellipsis is ill-formed.~~ [

Example:

```
template<class... Args>
void f(Args... args) {
    auto lm = [&, args...] { return g(args...); };
}
```

```

lm();

auto lm2 = [xs=std::move(args)...]_{ return g(xs...); };
lm2();
}

```

- end example]

It is likely that some additional wording change would be necessary in [expr.prim.lambda.capture]/6.

This would simplify all the code that currently relies on `std::tuple` just to solve this problem, in a way that we are already used to seeing pack expansion:

C++17 today	This proposal
<pre> template <class... Args> auto delay_invoke_foo(Args... args) { return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) { return std::apply([&](auto const&... args) -> decltype(auto) { return foo(args...); }, tup); }; } </pre>	<pre> template <class... Args> auto delay_invoke_foo(Args... args) { return [args=std::move(args)...]() -> decltype(auto) { return foo(args...); }; } </pre>

Acknowledgements

Thanks to T.C. for suggesting this solution and pointing out the history of the *init-capture* restriction.

Thanks to Richard Smith, John Spicer, and Daveed Vandevoorde for considering the viability of this change.

References

- [1] [N3610](#): "Generic lambda-capture initializers, supporting capture-by-move"
- [2] [N3648](#): "Wording Changes for Generalized Lambda-capture"
- [3] [A problem with generalized captures and pack expansion](#)
- [4] [CWG 1760](#): Access of member corresponding to *init-capture*